MACOS NATIVE APPLICATION BLUEPRINT • UNIVERSAL BINARY (APPLE SILICON & INTEL)

IMAGE + PDF UTILITY PRO FOR macOS

Production System Design, High-Speed Algorithms, Native Apple Frameworks & Human Interface Guidelines (HIG) Specification

Target Platform: macOS 13.0+ (Ventura, Sonoma, Sequoia) | **Architecture:** Universal (arm64 + x86_64) | **Execution Model:** 100% Local On-Device Processing | **Version:** 2.0 (macOS Edition)

100% Local	< 35 ms	arm64 + x86_64	Swift 6 + Metal
ZERO CLOUD UPLOAD / PRIVACY FIRST	50 KB IMAGE TARGET SEARCH (M-SERIES)	UNIVERSAL BINARY SUPPORT	HARDWARE-ACCELERATED ENGINE

1. Executive Summary & Web-to-macOS Transformation

This document translates the reference web-based Image and PDF Utility Platform (PI7-style workflow) into a high-performance, native **macOS Desktop & Menu Bar Application**. While the web architecture relied on Next.js, Amazon S3, SQS queues, and ECS Fargate workers to process files remotely, the macOS edition eliminates server costs, network latency, and privacy risks by executing **100% of image and PDF transformations locally on the user's Mac** using hardware-accelerated Apple frameworks.

1.1 Core Product Principles for macOS

- **Universal Mac Compatibility:** Compiled as a Universal Binary (*arm64* for Apple Silicon M1–M5 and *x86_64* for Intel Macs) targeting macOS 13.0+ so it runs smoothly across all modern MacBooks, iMacs, Mac minis, and Mac Studios.
- **Instant Hardware-Accelerated Speed:** Replaces web workers and cloud containers with Apple's **Accelerate (vImage)**, **Metal**, **Core Image**, **ImageIO**, **PDFKit**, and **Vision** frameworks.
- **macOS-First User Experience:** Follows Apple Human Interface Guidelines (HIG) with a clean 3-column `NavigationSplitView`, live before/after interactive canvas, Menu Bar Quick-Drop zone, Finder Quick Actions, and global keyboard shortcuts.
- **Strict Output Honesty & Verification:** Preserves source pixel dimensions and aspect ratio by default. Target KB/MB compression uses a deterministic hybrid secant-binary search over hardware encoders and verifies exact byte count, dimensions, and DPI metadata before export.
- **Complete Privacy & Sandboxing:** Operates inside the macOS App Sandbox. No internet connection is required, no user files ever leave the device, and temporary scratch buffers are purged deterministically.

1.2 Architectural Mapping: Web Stack vs. Native macOS Stack

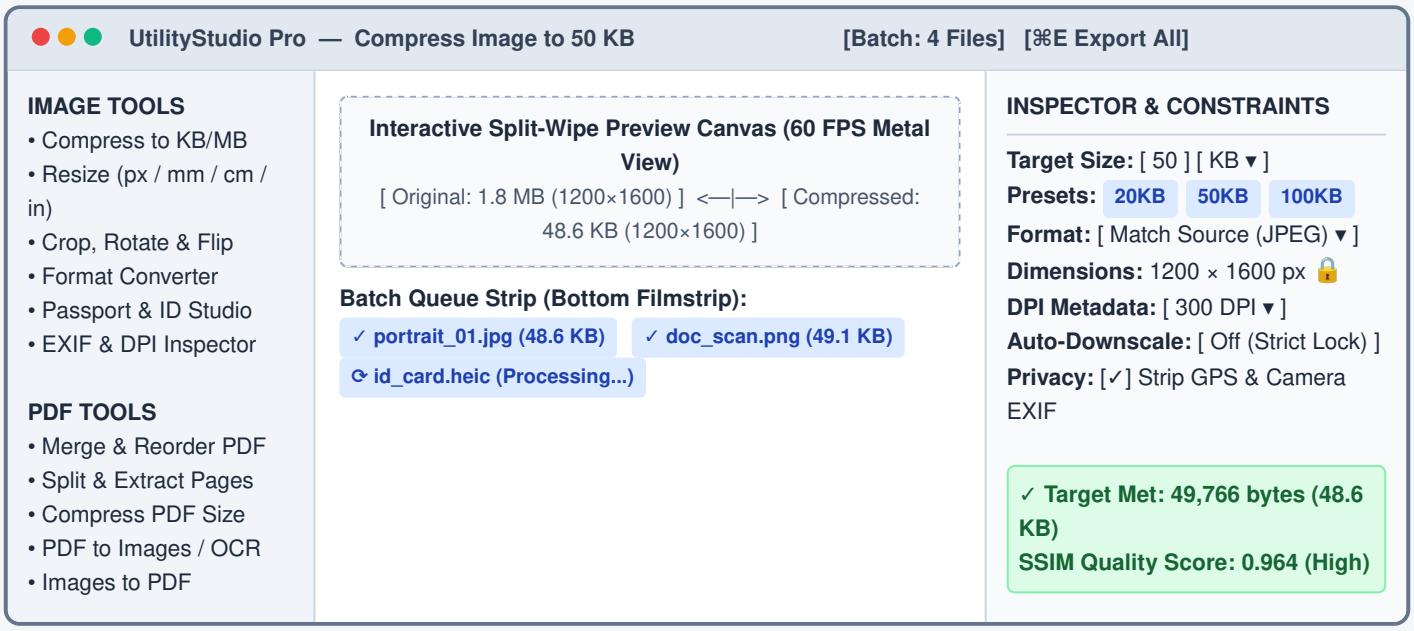
System Layer	Original Web Platform Stack	Native macOS Architecture (Recommended)
UI & Presentation	Next.js 16 App Router, React, Tailwind CSS, shadcn/ui	**SwiftUI 6 + AppKit Bridge** (`NSViewRepresentable` for high-FPS zoom/crop canvas, native macOS vibrancy & dark mode)
Concurrency & Queue	Web Workers + Amazon SQS + ECS Fargate containers	**Swift Structured Concurrency** (`Actor` isolation, `TaskGroup`) + `OperationQueue` bound to P-cores & memory pressure
Image Engine		

System Layer	Original Web Platform Stack	Native macOS Architecture (Recommended)
	OffscreenCanvas (client) + Node.js Sharp / libvips (server)	**ImageIO + Accelerate (vImage) + Core Image** / **Metal** + bundled **libvips** / **MozJPEG** static C libraries for ultra-low byte targets
PDF Engine	PDF.js (preview), pdf-lib (client), qpdf CLI (server)	**Apple PDFKit + Core Graphics (`CGPDFDocument`)** + embedded **libqpdf (C++ API)** for lossless structural optimization
OCR & Segmentation	Tesseract OCR worker + Python/ONNX segmentation	**Apple Vision Framework** (`VNRecognizeTextRequest` for OCR, `VNGeneratePersonSegmentationRequest` for Passport IDs)
Storage & State	Private S3 buckets (presigned URLs) + PostgreSQL + Zustand	**Security-Scoped Bookmarks** (local disk zero-copy I/O), **SwiftData** / **SQLite** (presets & history), **Keychain** (preferences/licenses)

2. Native macOS User Interface & UX Architecture (Apple HIG)

A great macOS utility must feel like a built-in part of the operating system—effortless for beginners yet powerful for daily professionals. The interface avoids cluttered web banners and confusing popups, focusing on immediate visual feedback, drag-and-drop simplicity, and keyboard-first efficiency.

2.1 Primary Workspace Layout (Three-Pane Split View)



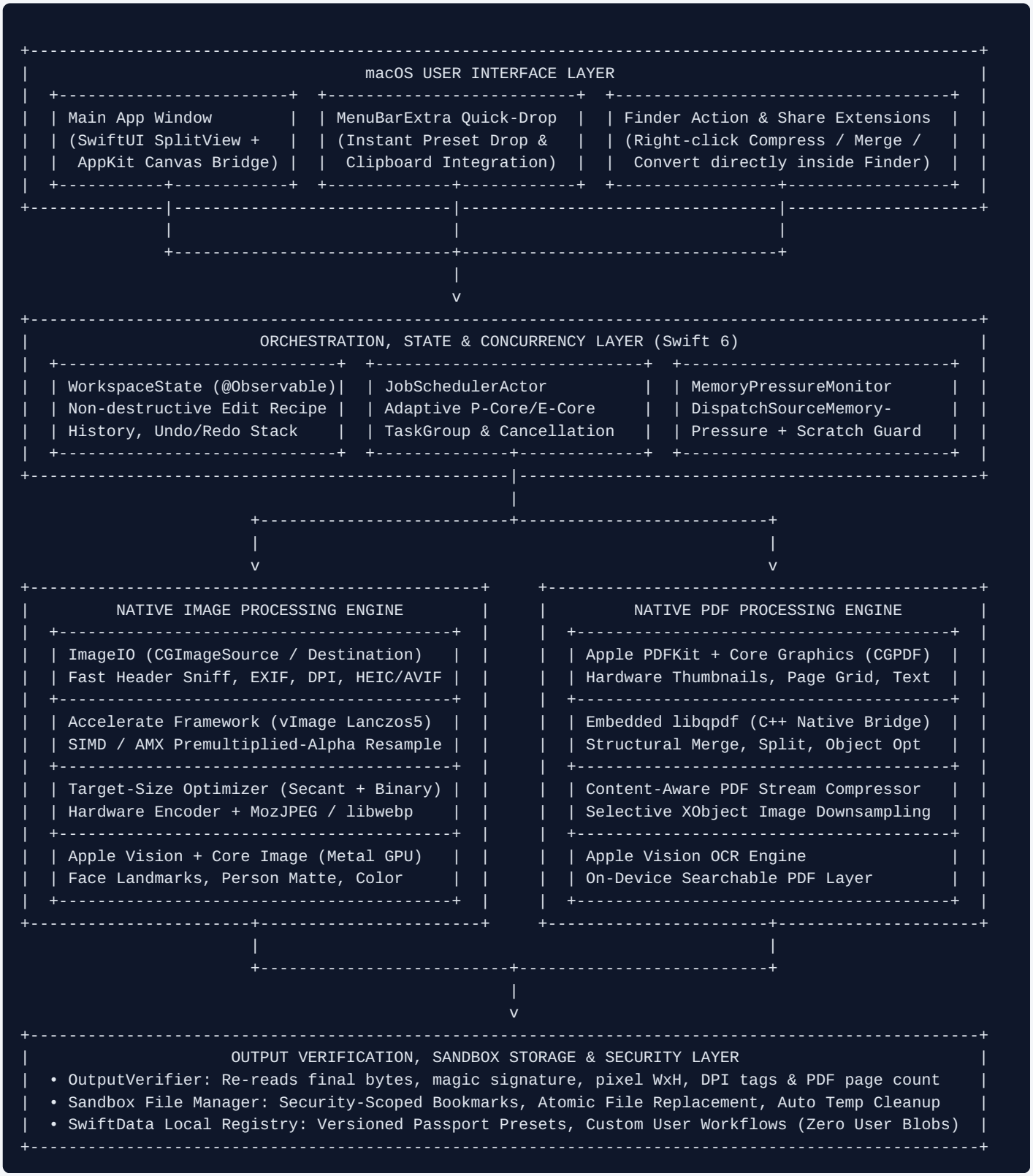
2.2 Key macOS Ergonomics & System Integrations

- **Menu Bar Quick-Drop Assistant (`MenuBarExtra`):** An optional lightweight icon in the macOS top menu bar. Users can drag a file or screenshot directly onto the menu bar icon, pick a favorite preset (e.g., *"Compress to 50 KB"* or *"Convert HEIC to JPG"*), and copy the processed file straight back to the clipboard or desktop in under 2 seconds.
- **Clipboard & Screenshot Workflow (⌘V / ⌘C):** Users can take a macOS screenshot (⌘⇧4 or ^⌘⇧4), press ⌘V inside the app to compress or crop it immediately, and press ⌘C to copy the optimized file back to `NSPasteboard`.

- **Finder Quick Actions & Services Extensions:** Right-clicking any image or PDF in Finder under *Quick Actions* exposes instant commands: *"Compress to Target KB"*, *"Merge Selected PDFs"*, and *"Convert to JPEG"* without even launching the main window.
- **Interactive Before/After Wipe & Loupe:** Built with a Metal-backed `MTKView` that supports smooth trackpad pinch-to-zoom (up to 800%), two-finger pan, a draggable split curtain comparing Original vs. Output, and a 100% zoom loupe for checking fine text or facial details in passport photos.
- **Honest Constraint Status Banner:** Clearly displays one of three states with color + icon + text (never color alone):
  1. **Target Met (Green Check):** Output is $\leq B_{max}$ with locked dimensions and high perceptual quality.
  2. **Target Met via Auto-Downscale (Amber Info):** Output met the byte target after permitted proportional resolution scaling.
  3. **Target Infeasible under Strict Lock (Red/Orange Actionable Alert):** Explains why (e.g., *"4000×3000 PNG cannot reach 20 KB without format change or downscaling"*) and provides one-click action buttons: *[Allow Auto-Downscale]* or *[Switch to WebP/JPEG]*.

3. High-Level System Architecture

The application is structured as a modular, multi-layered native macOS system with strict separation between the declarative UI layer, the concurrency orchestration layer, and the deterministic C/C++/Swift processing engines.



3.1 Why Process In-Process vs. XPC Helper Services?

- **Standard Operations (In-Process Actor Pool):** Image decoding, resizing, target-size binary search, and standard PDF merges execute inside background Swift Actors using zero-copy `vImage_Buffer` and `CFData` in-memory streams. This eliminates IPC serialization overhead and completes 50 KB image optimizations in **15–35 milliseconds** on Apple Silicon.
- **Untrusted / Complex Parsing (Isolated XPC Service):** Malformed PDFs, deep structural `libqpdf` rewrites, or huge 200-page OCR batches run inside a sandboxed **macOS XPC Helper Service** with restricted entitlements (no

network access, strict memory ceiling). If a corrupt third-party file triggers a parser crash or out-of-memory fault, the XPC service restarts transparently without crashing the main UI window.

4. Core Algorithms & Mathematical Specifications

To ensure the application works rapidly and produces superior visual quality compared to web tools, the following deterministic algorithms govern the processing engines.

Algorithm 1: Exact Target-Size Image Compression (Hybrid Secant + Binary Search with SSIM Floor)

Web tools often use slow linear loops (testing 92, 86, 80, 74...) that require 8–12 full encodes. On macOS, we model the rate-distortion curve where encoded file size $S(q)$ is approximately monotonic with respect to encoder quality $q \in [q_{min}, 1.0]$, using a fast 2-phase search in memory (`CFMutableData`):

1. **Initialization & Fast In-Memory Pipeline:** Decode source once into an orientation-normalized, premultiplied-alpha `vImage_Buffer` in the sRGB color space. Apply user crop and explicit resize first. Set target budget B_{max} (where $1\ KB = 1024\ bytes$ by default, with an optional $1000\text{-}byte$ decimal toggle).
2. **Phase 1 — Bracket & Secant Prediction (Iterations 1–3):**
   - Evaluate upper probe $q_{high} = 0.92$ and lower probe $q_{low} = q_{floor}$ (default 0.42).
   - If $S(q_{high}) \le B_{max}$, return immediately (1 encode needed!).
   - If $S(q_{low}) > B_{max}$, the target is unreachable at current dimensions; jump directly to Step 4 (Auto-Downscale or Infeasible Report).
   - Otherwise, estimate the next quality probe using log-size secant interpolation:

$$q_{next} = q_{low} + (q_{high} - q_{low}) \times [\ \ln(B_{max}) - \ln(S(q_{low}))\ ] / [\ \ln(S(q_{high})) - \ln(S(q_{low}))\ ]$$

3. **Phase 2 — Guarded Binary Refinement (Iterations 4–6 max):** Narrow the interval $[q_{low}, q_{high}]$ until $0.94 \times B_{max} \le S(q^*) \le B_{max}$ or $|q_{high} - q_{low}| < 0.01$. Always retain the highest-quality valid buffer $S(q^*) \le B_{max}$ to handle minor non-monotonic encoder steps.
4. **Step 4 — Scale-Space Fallback (Only if `allowDownscale == true`):** If $S(q_{floor}) > B_{max}$, compute dimensional scale ratio using the quadratic pixel relationship:

$$r = max(0.25, min(0.92, \sqrt{[\ B_{max} / S(q_{floor})\ ] \times 0.95}))$$

   Resample dimensions to $W' = round(W \times r)$, $H' = round(H \times r)$ via `vImageScale_ARGB8888` (Lanczos-5) and repeat Phase 1.
5. **Perceptual Quality Verification:** Compute hardware-accelerated Structural Similarity Index ($SSIM$) via Metal shader between source and candidate. If $SSIM < 0.82$, attach a visible quality warning in the Inspector.

Algorithm 2: High-Precision Physical Unit, DPI & Aspect-Locked Resizing

Unlike browser canvas exports that silently hardcode 96 DPI, the macOS engine maintains strict agreement between pixel geometry and embedded print resolution tags (EXIF `XResolution`/`YResolution`, JFIF density, and PNG pHYs pixels-per-meter):

$$P_{px} = round(L_{in} \times DPI) \quad | \quad P_{px} = round((L_{cm} \times DPI) / 2.54) \quad | \quad P_{px} = round((L_{mm} \times DPI) / 25.4)$$

$$PNG\ pHYs_{ppm} = round(DPI \times 39.37007874)$$

- **Premultiplied-Alpha Resampling:** Before calling `vImageScale_ARGB8888` with `kvImageHighQualityResampling` (Lanczos-5 kernel), alpha channels are premultiplied via `vImagePremultiplyData_RGBA8888` and unpremultiplied after scaling. This eliminates dark fringes/halos around transparent PNG and WebP logos.
- **Normalized Non-Destructive Coordinate Model:** Crop boxes are stored as normalized tuples $(x, y, w, h) \in [0, 1]^4$ after EXIF orientation normalization (`CGImagePropertyOrientation`). Deterministic pipeline order: **EXIF Orientation Normalize → User Rotate/Flip → Crop → Lanczos Resize → Color Profile Convert (sRGB) → Encode → DPI Metadata Injection**.

Algorithm 3: Passport & Official ID Studio (Apple Vision Face Framing + Matte Segmentation)

Creates compliant passport, visa, PAN, UPSC/SSC, and international ID photos completely offline:

1. **Biometric Landmark Detection:** Run `VNDetectFaceLandmarksRequest` on the input photo to detect the head bounding box, chin point, crown estimation, eye-line vector, and roll angle θ.
2. **Auto-Level & Rule-Driven Crop Box:** If head tilt $|\theta| > 1.5°$, offer one-click straighten. Automatically compute the crop rectangle so the head height occupies the preset's required ratio (e.g., $60\%–70\%$ of frame height for 35×45 mm or 2×2 inch presets) and eyes align with the jurisdiction's eye-line band.
3. **Optional Solid Background Replacement:** Execute `VNGeneratePersonSegmentationRequest` (`.accurate` quality level) coupled with Core Image guided edge refinement (`CIBlendWithMask`) to replace cluttered backgrounds with approved solid white (#FFFFFF), off-white, or light blue without altering facial identity or skin features.
4. **Print Sheet Generator (A4 / 4×6 inch):** Generates an exact 300 DPI printable sheet with configurable photo count (e.g., 8, 12, or 36 copies), bleed gaps, and subtle hairline cut marks.

Algorithm 4: Content-Aware PDF Compression & Structural Merge Engine

Web compressors often rasterize entire PDFs into blurry images. Our native engine preserves vector text, fonts, and links while selectively optimizing heavy streams:

1. **Phase 1 — Structural Inspection:** Parse the PDF tree via `CGPDFDocument` and `libqpdf`. Inventory page count, embedded font subsets, vector paths, XObject raster images (recording their native pixel dimensions vs. rendered point bounding box on the page), digital signatures, and AcroForms.
2. **Phase 2 — Effective DPI Calculation per Embedded Image:** For each image XObject i rendered inside a page bounding box of width W_{pt} points ($1\ inch = 72\ points$):

$$DPI_{eff,\,i} = W_{px,\,i} / (W_{pt,\,i} / 72.0)$$

3. **Phase 3 — Selective XObject Re-Encoding:**
   - *Lossless Mode:* Run `libqpdf` object stream compression, deduplicate identical font/image objects, and strip unreferenced resources. Zero change to pixel streams.
   - *Balanced Mode:* Only downsample image XObjects where $DPI_{eff,\,i} > 200$ down to $150\ DPI$ using Accelerate Lanczos scaling and re-compress color/grayscale streams at JPEG quality 0.75; keep monochrome 1-bit scans in lossless CCITT G4 / Flate. Vector text and fonts remain 100% untouched and crisp.
   - *Target KB Mode (Iterative Budget):* Binary-search the target effective DPI $\in [96, 220]$ and image quality $q \in [0.45, 0.85]$ across embedded image streams until total output PDF size $\leq B_{max}$.
4. **Preflight Safety Warnings:** If the PDF contains a cryptographic digital signature, display a native preflight sheet warning that re-saving or merging will invalidate the signature. Detect duplicate AcroForm field names during PDF merge and namespace them automatically so form data never collides.

5. Complete Feature & Native Framework Matrix

Tool Module	Supported Capabilities & Presets	macOS Native Frameworks & Engine Pipeline
Image Compress to KB / MB	Presets (10 KB, 20 KB, 50 KB, 100 KB, 500 KB, 1 MB) + custom byte ceiling, strict dimension lock, SSIM preview	`ImageIO` + `vImage` + Hybrid Secant-Binary Search over hardware JPEG/HEIC/WebP + bundled static `libmozjpeg`
Resize & DPI Studio	Resize by px, mm, cm, inch; aspect ratio lock; Contain / Cover / Pad modes; explicit 72/96/150/300/600 DPI write	`vImageScale_ARGB8888` (Lanczos-5 premultiplied alpha) + `CGImageDestination` EXIF/JFIF/PNG `pHYs` injector
Crop, Rotate, Flip & Effects	Free crop, 1:1, 4:3, 16:9, 35:45 ratios, circle mask, straighten, watermark, border, blur/pixelate sensitive areas	`MTKView` (Metal 60 FPS interactive canvas) + `CoreImage` filter chain (`CILanczosScaleTransform`, `CIPixellate`)
Format Converter	HEIC, AVIF, WebP, PNG, JPG/JPEG, TIFF, GIF, ICO, SVG rasterization; transparency fill picker for PNG→JPG	Native macOS hardware decoders/encoders via `UTType` & `CGImageDestination`; frame policy prompt for animated files
Passport & ID Photo Maker	Versioned country & exam presets (India Passport, PAN, UPSC/SSC signature, US 2×2", UK/EU 35×45mm), A4/4×6 sheet	`Vision` (`VNDetectFaceLandmarksRequest` + `VNGeneratePersonSegmentationRequest`) + Core Graphics PDF/Print sheet

Tool Module	Supported Capabilities & Presets	macOS Native Frameworks & Engine Pipeline
PDF Merge, Split & Page Grid	Multi-PDF drag reorder, page-level grid rearrange, delete, rotate 90°/180°, crop page boxes, split by ranges (e.g. 1-3, 5)	`PDFKit` thumbnail grid + `libqpdf` C++ bridge for lossless page tree assembly without rasterization
PDF Compress & OCR	Lossless structural, Balanced, Smallest, or Target KB; PDF↔JPG/PNG at custom DPI; offline searchable PDF OCR	`CGPDFDocument` XObject parser + `libqpdf` + `Vision` (`VNRecognizeTextRequest` multi-language offline OCR)
Batch Queue & ZIP Export	Process 100+ files at once with shared settings or per-file overrides; export to folder or single compressed ZIP	`TaskGroup` bounded by P-core count & memory pressure + Apple `Archive` / `Compression` framework

6. Concurrency, Low-RAM Memory Model & Universal Binary Tuning

A 6000×4000 16-bit image or a 500-page scanned PDF can easily consume gigabytes of RAM if decoded naively. To guarantee smooth performance on both an 8 GB Intel MacBook Air and a 64 GB M5 MacBook Pro, the engine enforces strict memory budgets:

- **Two-Tier Image Decoding (Thumbnail vs. Export Pipeline):** The UI never loads full-resolution bitmaps into SwiftUI views. Preview thumbnails are decoded via `CGImageSourceCreateThumbnailAtIndex` with `kCGImageSourceThumbnailMaxPixelSize = 1600` and `kCGImageSourceCreateThumbnailFromImageAlways = true`. Full-resolution buffers are allocated only inside the background export worker and wrapped in `autoreleasepool { ... }` blocks so memory is reclaimed immediately after each file completes.
- **Tile-Based Processing for Ultra-High-Res Rasters:** For images exceeding $25\ Megapixels$, `vImage` and `CIContext` process the raster in horizontal tiles rather than allocating a single contiguous multi-hundred-megabyte RGBA buffer.
- **Adaptive Batch Concurrency Governor:**
  - Reads hardware topology via `sysctlbyname("hw.perflevel0.physicalcpu")` to identify Apple Silicon Performance (P) cores vs. Efficiency (E) cores.
  - Sets maximum concurrent heavy jobs to $N_{workers} = max(1, min(P_{cores} - 1, 4))$.
  - Subscribes to `DispatchSource.makeMemoryPressureSource(eventMask: [.warning, .critical])`. If macOS reports `.warning` memory pressure during a 100-file batch, the `JobSchedulerActor` automatically throttles concurrency to 1 active file at a time until pressure subsides.

7. Xcode Project & Modular Swift Package Architecture

Instead of a web monorepo (Turborepo/pnpm), the macOS codebase uses a clean **Xcode Workspace + Local Swift Package Manager (SPM)** modular architecture. This enforces strict compile-time boundaries and lets you test the core engines in isolation without launching the UI.

```
UtilityStudioMac/
├── UtilityStudioMac.xcodeproj                # Main macOS Application Target, MenuBarExtra &
Extensions
├── App/
│   ├── UtilityStudioApp.swift                # @main entry point, WindowGroup + MenuBarExtra scenes
│   ├── Navigation/                           # Sidebar router, tool switching, deep links & Cmd+K
palette
│   ├── Views/
│   │   ├── Workspace/                        # ThreePaneSplitView, DropzoneOverlay,
BatchFilmstripView
│   │   ├── Canvas/                           # MetalComparisonView (NSViewRepresentable),
CropOverlayView
│   │   ├── Inspector/                        # ConstraintBadgeView, TargetSizeControls,
PresetPickerView
│   │   └── PDFGrid/                          # PDFPageThumbnailGridView, PageRangeSplitSheet
│   └── Extensions/
│       ├── FinderQuickActionExtension/       # NSExtension for right-click Finder Compress / Merge /
Convert
│       └── ShareExtension/                   # System Share Sheet receiver
├── Packages/
│   ├── CoreContracts/                        # Typed ToolDefinitions, EditRecipe, JobState,
VerificationReport
│   ├── ImageEngineKit/                       # ImageIO, vImage Lanczos, Secant-Binary Optimizer, DPI
Injector
│   ├── PDFEngineKit/                         # PDFKit, CGPDF XObject Downsampler, libqpdf C++ Wrapper
│   ├── VisionStudioKit/                      # Passport Face Landmarks, Person Segmentation, Offline
OCR
│   ├── FileSecurityAndVerifyKit/             # Magic-byte sniffer, Dimension/Bomb guards,
OutputVerifier
│   └── PresetRegistryKit/                    # Versioned JSON/SwiftData Passport & Print Presets
├── Vendor/
│   ├── libqpdf.xcframework                   # Universal (arm64 + x86_64) static build of qpdf
(Apache-2.0)
│   └── libmozjpeg.xcframework                # Universal (arm64 + x86_64) static build for ultra-low
KB JPEGs
└── Tests/
    ├── ImageEngineTests/                     # Target-size invariants, EXIF rotation, DPI & alpha
halo tests
    ├── PDFEngineTests/                       # Page manifest order, AcroForm collision, compression
tests
    └── UIPerformanceTests/                   # XCTest memory ceiling, 60 FPS canvas & batch
cancellation tests
```

7.1 Core Swift Concurrency & Verification Contracts

```
public struct ImageCompressionOptions: Sendable, Codable {
    public var maxBytes: Int                      // e.g., 51_200 for 50 KiB
    public var outputFormat: UTType               // .jpeg, .png, .webP, .heic
    public var targetWidth: Int?                  // nil = keep source width
    public var targetHeight: Int?                 // nil = keep source height
    public var lockDimensions: Bool = true
    public var allowAutoDownscale: Bool = false
    public var targetDPI: Int? = 300              // Explicitly written to EXIF/JFIF/pHYs
    public var qualityFloor: Float = 0.42
    public var stripSensitiveMetadata: Bool = true // Removes GPS, camera serial & creator tags
}

public struct VerifiedOutputReport: Sendable {
    public let outputURL: URL
    public let actualBytes: Int
    public let verifiedUTType: UTType
    public let pixelWidth: Int
    public let pixelHeight: Int
    public let verifiedDPI: Int?
    public let ssimScore: Float?
    public let targetMet: Bool
    public let dimensionsChanged: Bool
    public let warnings: [ProcessWarning]
}

// Strict Invariant checked by OutputVerifier before enabling Save / Copy / Drag-Out:
// 1. If report.targetMet == true, then report.actualBytes <= options.maxBytes.
// 2. If options.lockDimensions == true && !options.allowAutoDownscale, output WxH == requested WxH.
// 3. Verified DPI is read back from disk via CGImageSourceCopyPropertiesAtIndex, never assumed from UI state.
```

8. Security, Privacy, Sandboxing & App Store Compliance

> **PRIVACY-FIRST LOCAL ARCHITECTURE & EXPORT COMPLIANCE**
>
> Because all image and PDF operations execute 100% locally on the user's Mac, the app collects **zero user files, zero telemetry on document contents, and requires no cloud upload servers**. For App Store Connect submission, the Privacy Nutrition Label qualifies for **"Data Not Collected"**, and standard OS/HTTPS calls qualify under encryption export compliance exemption (`ITSAppUsesNonExemptEncryption = NO` if only standard OS APIs are used).

- **macOS App Sandbox & Entitlements:**
  - `com.apple.security.app-sandbox = true`
  - `com.apple.security.files.user-selected.read-write = true` (Grants access only to files/folders the user explicitly drops, opens via `NSOpenPanel`, or saves via `NSSavePanel`).
  - **Security-Scoped Bookmarks:** If a user configures a default export folder (e.g., `~/Downloads/Optimized`), the app stores a security-scoped bookmark in `UserDefaults` so subsequent exports require zero extra permission prompts.
- **Magic-Byte Sniffing & Decompression Bomb Protection:** Before full decode, `FileSecurityAndVerifyKit` inspects the first 64 bytes of the file stream and reads header dimensions via `CGImageSourceCopyPropertiesAtIndex` (which does not decode raster pixels). Files exceeding configurable

pixel ceilings (e.g., ***150 MP*** or malformed ZIP/PDF object trees) are safely rejected with a clear native alert before allocating memory.

- **Deterministic Scratch Cleanup:** Temporary intermediate files are written exclusively to `FileManager.default.temporaryDirectory` inside a per-job UUID subfolder with `NSFileProtectionCompleteUntilFirstUserAuthentication` and purged inside a `defer { try? FileManager.default.removeItem(at: jobTempDir) }` block on completion, cancellation, or app launch.
- **Open-Source License Compliance (App Store Safe):**
  - Apple frameworks (`SwiftUI`, `Accelerate`, `Metal`, `ImageIO`, `PDFKit`, `Vision`): Built into macOS.
  - `libqpdf`: **Apache License 2.0** — fully permissive for commercial Mac App Store distribution.
  - `MozJPEG` / `libwebp`: **BSD 3-Clause License** — fully permissive for static linking in commercial apps.
  - *Strict Exclusion:* **Ghostscript (AGPL)** is explicitly excluded to avoid copyleft viral licensing issues on the Mac App Store.

9. Phased Engineering Roadmap & QA Acceptance Matrix

Phase	Core Deliverables	Exit Gate & Automated Verification Criteria
Phase 1: Core Image Engine & UI Shell	Xcode workspace, 3-pane `NavigationSplitView`, Metal 60 FPS split-wipe preview, `ImageIO` + `vImage` resize, Crop/Rotate, Format Converter, Target KB Secant-Binary Optimizer, `OutputVerifier`.	50 KB target compression completes in < 50 ms on Apple Silicon; 100% pass rate on EXIF orientation, premultiplied alpha, and DPI metadata read-back tests.
Phase 2: PDF Engine & Page Grid	`PDFKit` interactive page grid (reorder, rotate, delete, split), embedded `libqpdf` lossless merge, Content-Aware XObject PDF compressor, AcroForm & signature preflight warnings.	20-file PDF merge preserves selectable vector text and bookmarks; scanned PDF compression reduces size by 60–80% while keeping text layers crisp.
Phase 3: Passport Studio & Batch Queue	Versioned country/exam preset database, `Vision` face landmark framing guide & person segmentation background fill, A4/4×6 300 DPI print sheet generator, multi-file batch queue + ZIP export.	35×45 mm @ 300 DPI exports verified at exact 413×531 px with 300 DPI tags; 100-file batch runs within a 600 MB peak RAM envelope under memory-pressure tests.
Phase 4: macOS Polish & Distribution	`MenuBarExtra` Quick-Drop icon, Clipboard ⌘V/⌘C workflow, Finder Quick Action extension, offline `Vision` OCR, Hardened Runtime notarization & Mac App Store / TestFlight build.	Universal Binary (`arm64` + `x86_64`) passes Apple Notarization and App Sandbox validation with zero memory leaks in Xcode Instruments.

9.1 Definition of Done (Quality Gate for Every Tool)

1. **Zero Main-Thread Blocking:** The macOS UI maintains a locked 60/120 Hz frame rate on ProMotion displays while heavy batch jobs execute in background actors.
2. **Four-Way Input Support:** Every tool accepts files via (a) Drag-and-Drop onto window or Dock/Menu Bar icon, (b) Native File Picker (⌘0), (c) Paste from Clipboard (⌘V), and (d) Finder Right-Click Quick Action.

3. **Independent Output Verification:** The *Export / Save / Copy* button only transitions to the green ready state after `OutputVerifier` re-opens the encoded output stream and confirms exact byte ceiling ($\leq B_{max}$), pixel dimensions, DPI tags, and valid container structure.